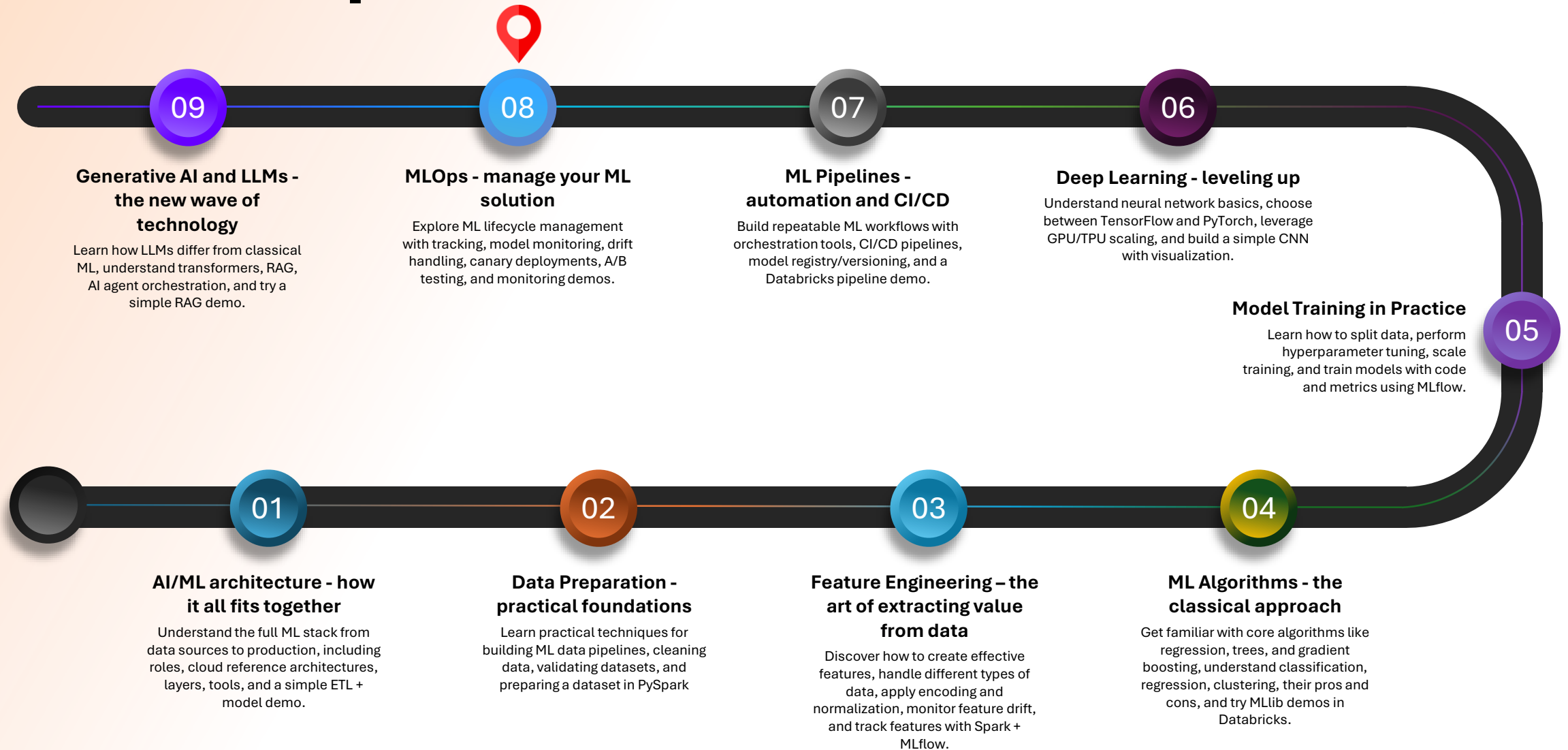


MLOps - manage your ML solution

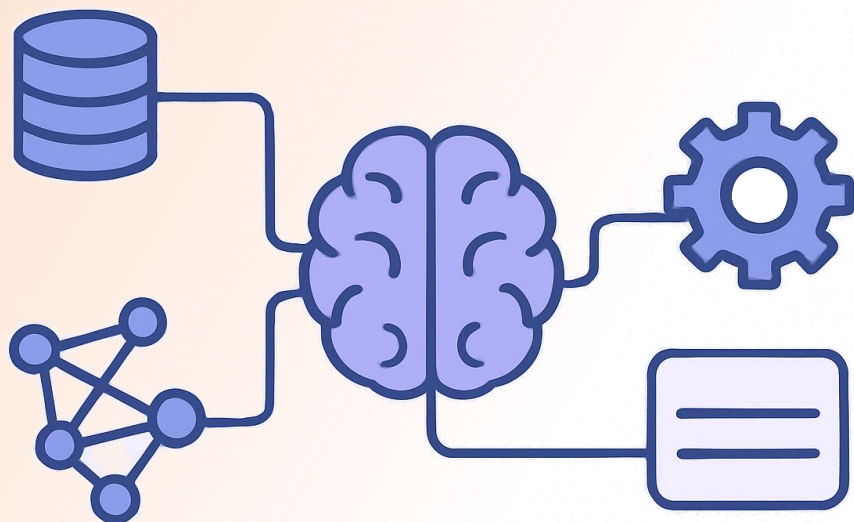
Maciej Kępa

Roadmap

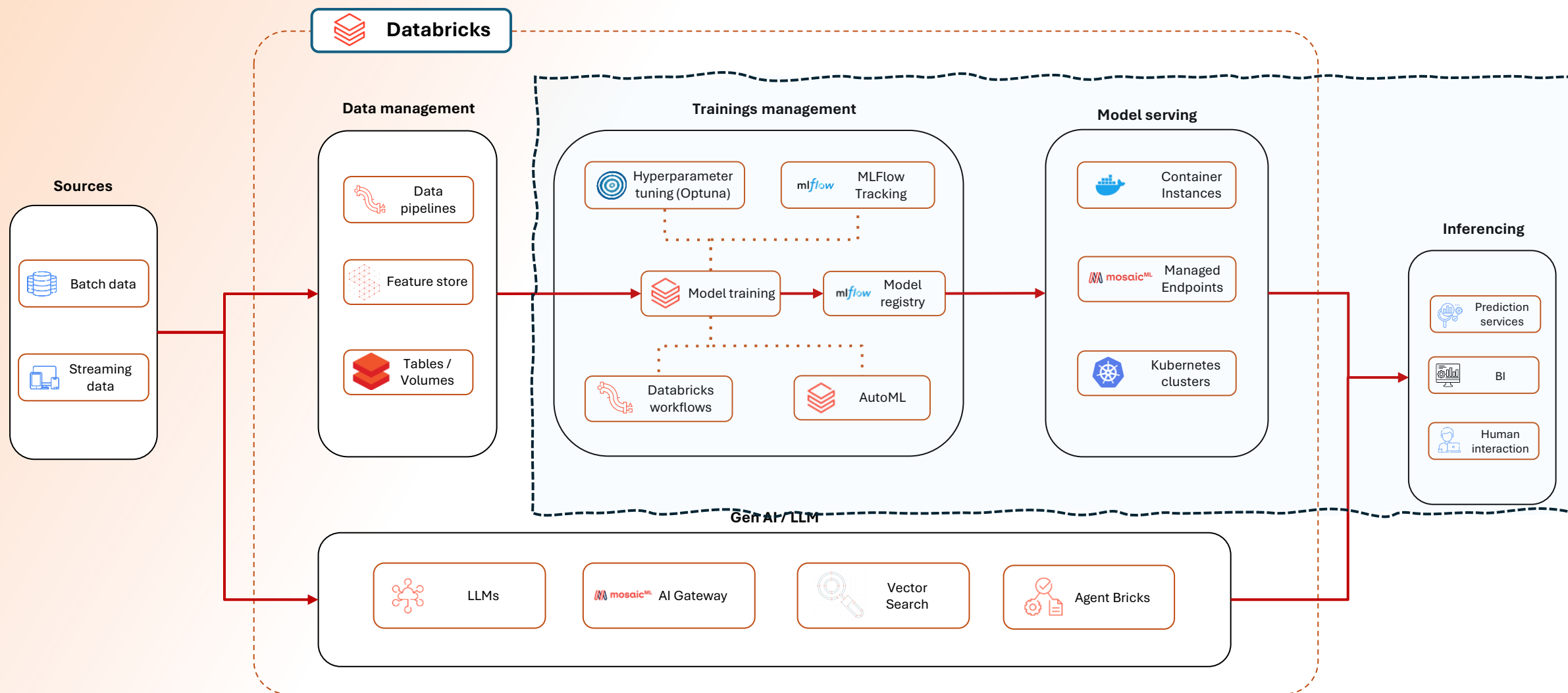


Agenda

1. Introduction
2. MLOps and Model as a Service
3. Monitoring and environment management
4. Safe deployment strategies
5. Workshop: environment management
6. Workshop: model monitoring
7. Workshop: canary release
8. Best practices



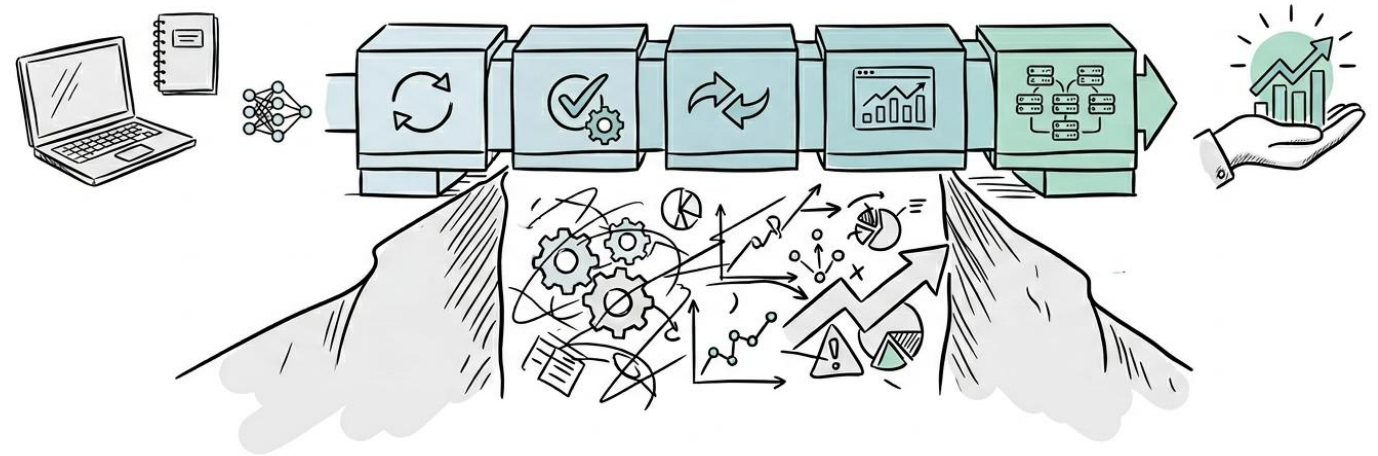
Introduction

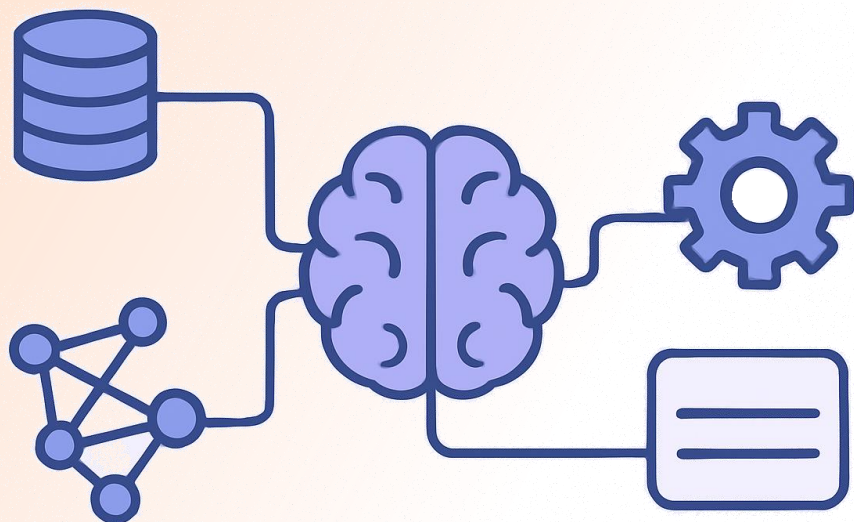


Beyond the notebooks

Training an ML model is just the beginning of your journey. Productionizing ML introduces significant complexity around reproducibility, scalability, and reliability.

The gap between a trained model and a valuable product is where MLOps bridges the divide, ensuring your ML solutions deliver consistent business value.





MLOps and Model as a Service

MLOps principles

MLOps transforms ML from an experimental science into a reliable engineering discipline, ensuring your ML solutions meet production standards.

Reproducibility

Reproducibility means every experiment, training run, and model version can be recreated with the same inputs and environment.

This matters because reproducibility is the foundation of experimentation, debugging and compliance.

Continuous Improvement

Continuous improvement makes ML a living system instead of a one-time release.

Instead of waiting for occasional manual reviews, teams can learn from every deployment, every test, and every production signal.

Collaboration

Collaboration keeps ML work visible and coordinated across roles.

Best practices break down silos between data scientists and engineers by turning ML into a shared workflow rather than a handoff problem.

Trustworthiness

Trustworthy ML systems are built on data and evidence, not assumptions.

Trust is what makes ML usable at scale: it gives teams the assurance to deploy, maintain, and rely on models in business-critical settings.

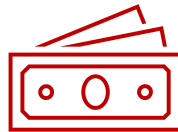
The silent killer - technical debt

In machine learning, technical debt often builds quietly. A model may work well today, but without disciplined tracking, versioning, and lineage, the cost of maintaining it grows rapidly as experiments multiply, data changes, and teams move on.



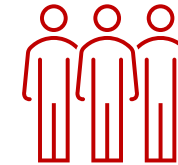
Time cost

Hours are lost debugging failed deployments, retraining models that should have been reproducible, and recreating experiments that were never properly saved. Small tracking gaps compound into major delays when a deadline or incident arrives.



Business cost

Poor model performance can affect revenue, customer trust, and operational efficiency. Failed deployments may interrupt critical workflows, while weak governance or missing audit trails can expose the organization to compliance and regulatory issues.



Team cost

Technical debt creates friction between data science, engineering, and operations teams. Knowledge gets trapped in individual notebooks or personal memory, silos deepen, and productivity falls as people spend more time explaining past decisions than building new value.

Model as a Service

Model-as-a-Service approach has recently become the standard, where models are exposed through scalable endpoints or integrated directly into analytical workflows. The goal is not just to build a model that works once, but to run it reliably in production and keep improving it over time.



Multi-dimensional Versioning

Linking the model artifact to a specific snapshot of the training data, the code hash, and the runtime environment specification ensures full reproducibility and change control.



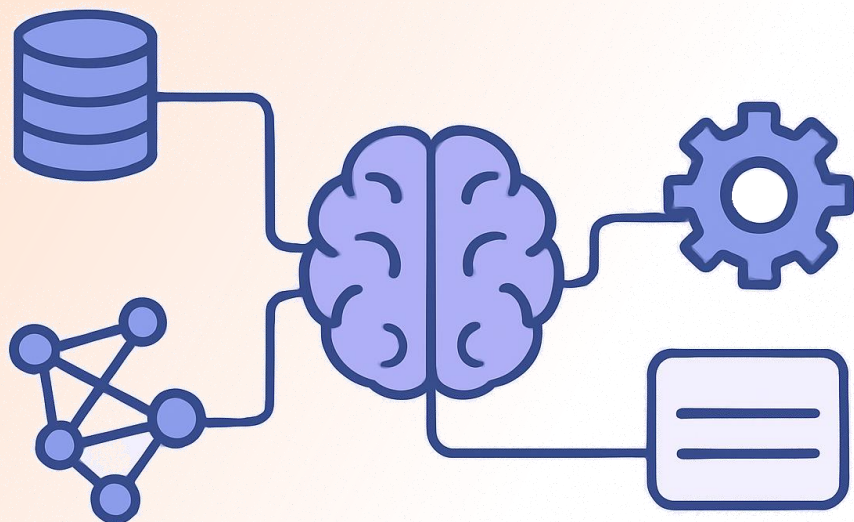
Quality Gates

Automated regression tests and statistical validation block models that do not meet the defined quality thresholds before they are allowed into production.



Feedback Loops

Collecting data on real-world outcomes (*ground truth*) makes it possible to retrain the model later and systematically improve its effectiveness.



Monitoring and environment management

Environment management

Environment management means keeping the training and serving environments aligned so a model behaves the same way in development, validation, and production. That includes the operating system, Python runtime, libraries, package versions, feature definitions, and even how data is preprocessed.

A model trained in one environment should see the same code paths, dependencies, and runtime assumptions when it is deployed. If feature logic, serialization libraries, or numerical packages differ, the same input can produce different outputs.

Docker containers

Package the model, application code, and runtime dependencies into a reproducible container so every environment starts from the same base image.

Dependency planning

Lock exact library versions in files such as *requirements.txt* or conda specifications to prevent surprise upgrades from changing behavior.

Version management

Standardize the interpreter version across training and serving to avoid incompatibilities in syntax, serialization, or library support.

Consistent environments reduce deployment failures, prevent subtle prediction errors, and make models easier to trust and reproduce.

Model monitoring

Implementing robust monitoring for machine learning models requires a clear strategy encompassing key metrics, statistical methods, and reliable infrastructure to ensure model health and performance.

Effective monitoring tracks model performance and behavior in real-world scenarios

- **Latency & Throughput** - measure prediction response times and the volume of requests processed per second to ensure service responsiveness and scalability.
- **Error rates** - track inference errors (e.g., failed predictions, invalid inputs) and system errors to identify operational issues.
- **Prediction distributions** - monitor changes in output predictions (e.g., score ranges, class probabilities) to detect shifts in model behavior or data patterns.

Alerting Thresholds

Set specific, actionable thresholds for key metrics that indicate a problem, such as sustained high latency or significant prediction drift.

Escalation Policies

Establish clear procedures for who to notify, and when, based on the severity and impact of an alert.

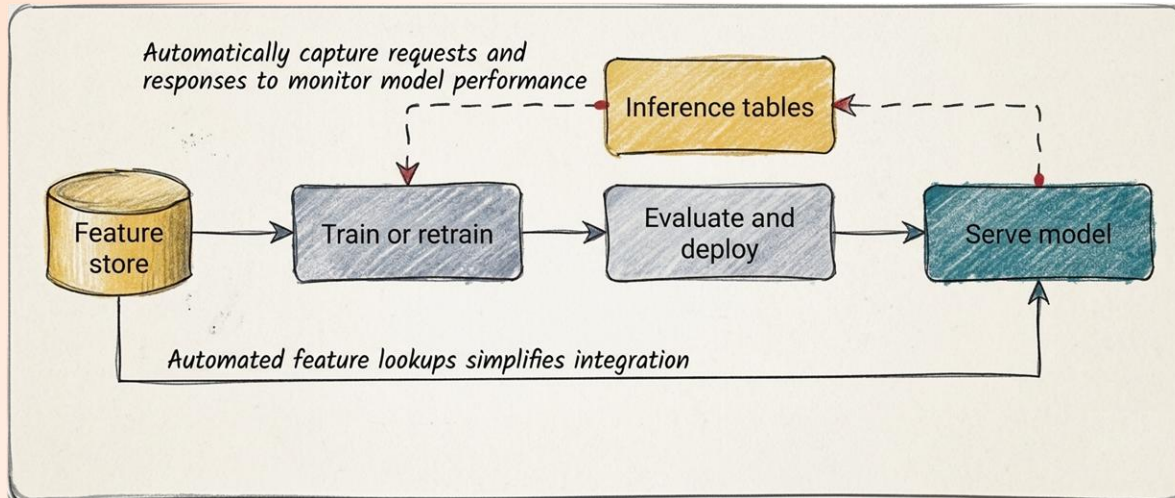
Incident Runbooks

Provide detailed, step-by-step guides for diagnosing and resolving common issues identified by monitoring alerts.

📌 A proper implementation of human-in-the-loop approach acts also as your model monitoring! A human is one of the most sensitive sensors you can install in your system.

Inference tables

Inference tables are a modern approach for robust MLOps, capturing every prediction a deployed model makes, along with its associated inputs and metadata. These tables become a rich historical record, enabling comprehensive monitoring and driving continuous improvement for your AI systems.



Capturing Predictions

Inference tables store model predictions, probabilities, confidence scores, and raw input data, creating a verifiable record of model behavior in production.



Dynamic Performance Dashboards

Visualize key metrics like model accuracy, precision, recall, data drift, and prediction distribution over time, allowing for proactive detection of performance degradation.



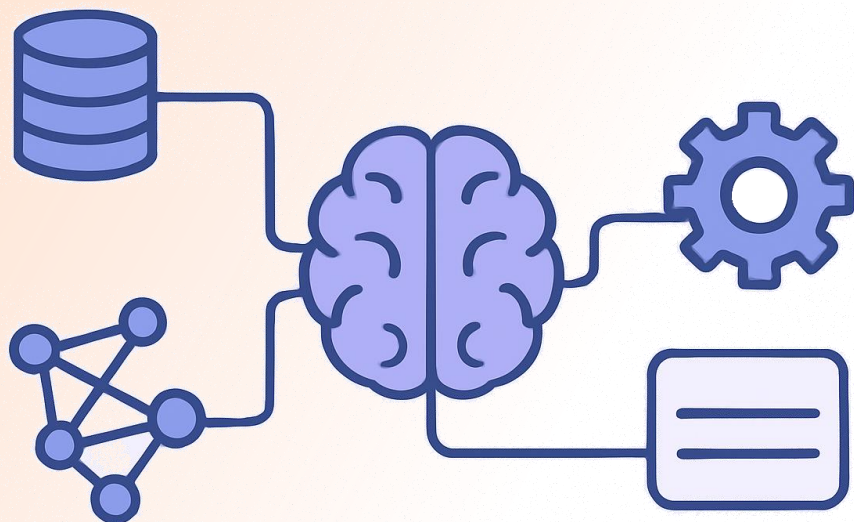
Alerts & anomaly detection

Configure automated alerts for significant performance drops or unusual prediction patterns, using anomaly detection to flag subtle issues before they impact business outcomes.



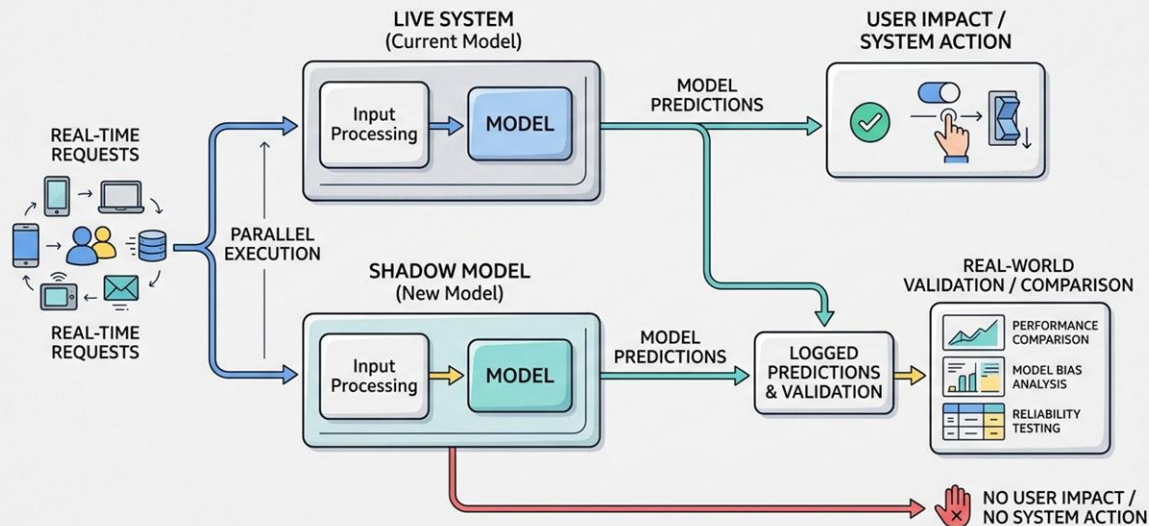
Integrated Feedback Loops

Join ground truth data back to inference records to automatically calculate true model performance, informing targeted retraining and systematically improving model effectiveness.



Safe deployment strategies

Shadow deployment



How It Works

Run the new model in parallel with the live system. Its predictions are **logged but never acted upon**, allowing real-world validation without user impact.

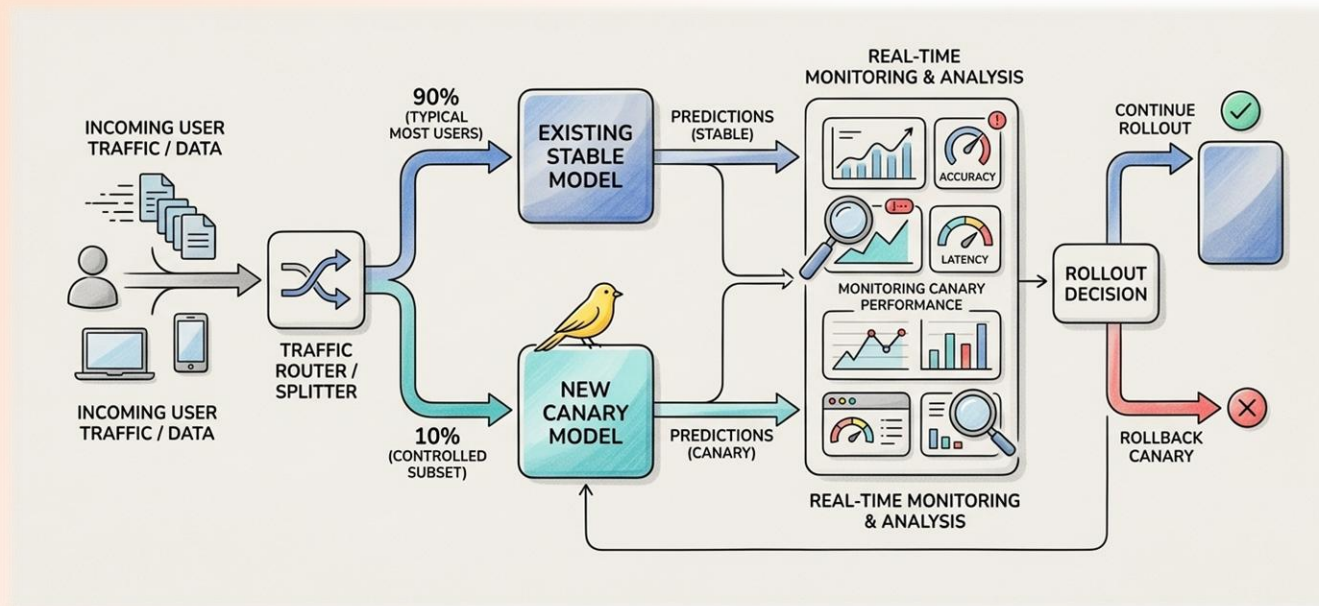
Key Benefit

Detect regressions early and reduce risk before changes reach all users, making it easier to scale safely with real-world data.



A recommendation engine runs a new algorithm in shadow mode, comparing its suggestions against the current live system before any switch is made.

Canary deployment



How It Works

Deploy a new model to a small, controlled subset of traffic while most users continue to be served by the existing stable model. The performance of this new "canary" model is monitored in real-time.

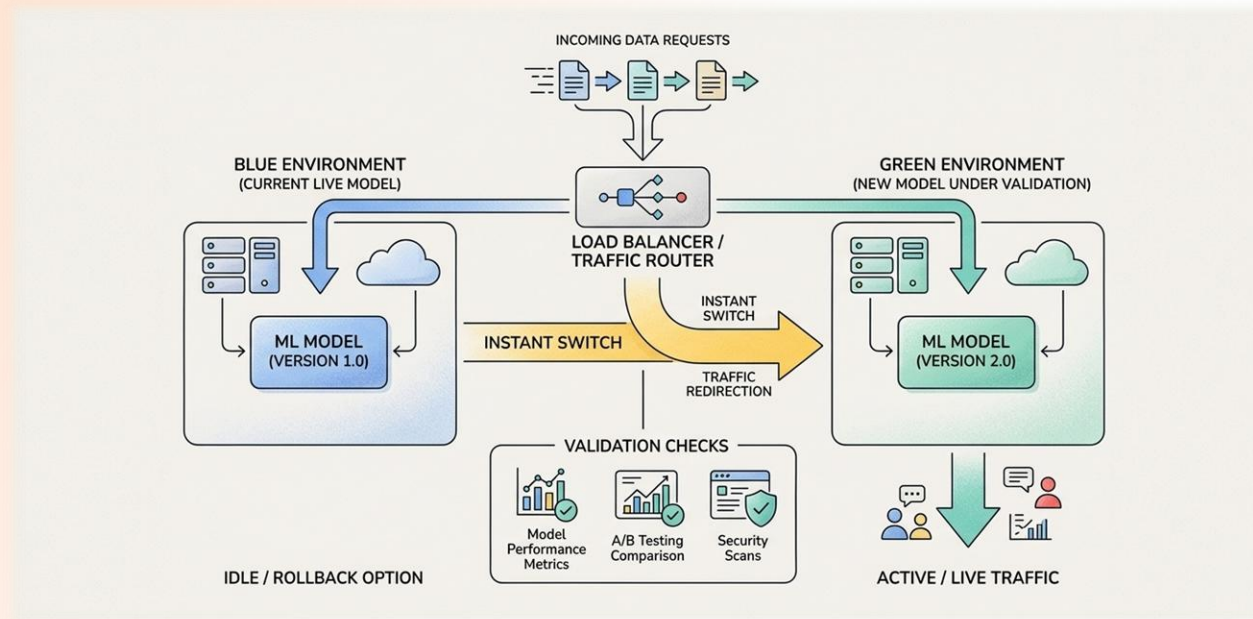
Key Benefit

Test with live traffic and real data distributions - zero risk to users or business outcomes.



A new fraud detection model is initially deployed to only 5% of transactions. Performance is monitored closely before expanding to 25%, 50%, and eventually 100%.

Blue/green deployment



How It Works

Maintain two identical production environments: **Blue** (current live model) and **Green** (new model under validation). Once Green passes all checks, a load balancer switch redirects all traffic instantly.

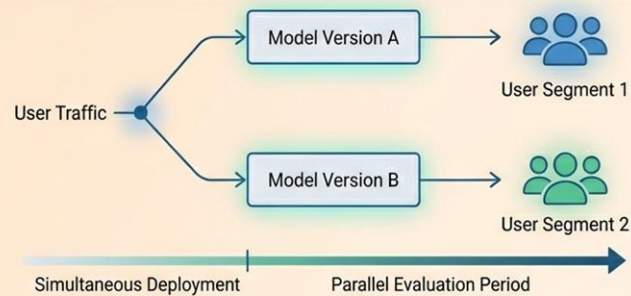
Key Benefit

Near-instantaneous rollback - simply switch traffic back to Blue if issues arise.



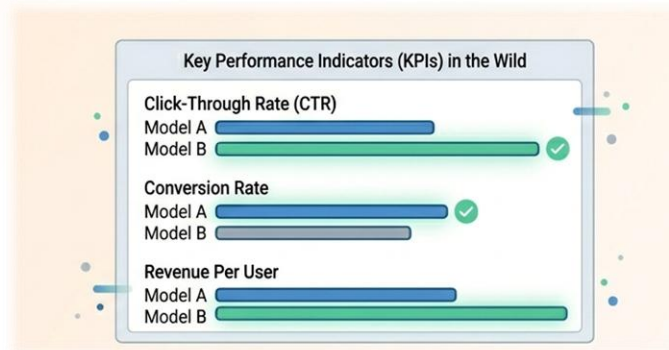
A new pricing model is deployed to Green. After validation, the load balancer is updated to direct all traffic to Green in seconds.

A/B testing



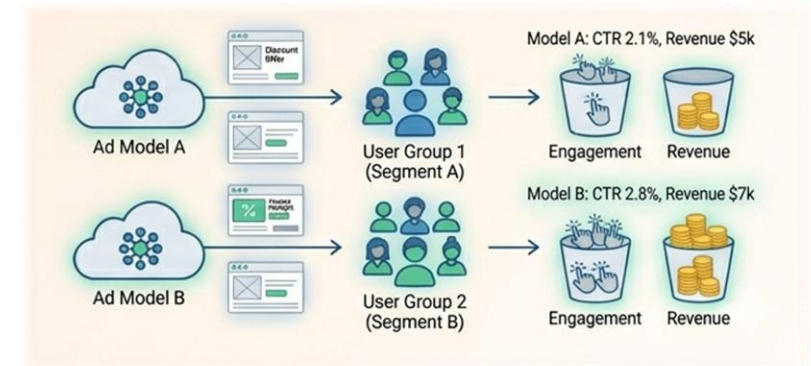
Simultaneous Comparison

Deploy multiple model versions to distinct user segments at the same time, enabling direct, apples-to-apples performance comparison.



Metrics-Driven Decisions

Compare click-through rates, conversion rates, or other KPIs to determine which version objectively performs better in the wild.



Real-World Example

Two versions of a personalized ad model are shown to different user groups to identify which drives higher engagement and revenue.

Rollback triggers

Define clear conditions for automatic rollback to protect business operations and user experience.

Performance Drop

Significant decrease in key metrics like accuracy, precision, or recall

Latency Increase

Response time exceeding acceptable thresholds impacting user experience

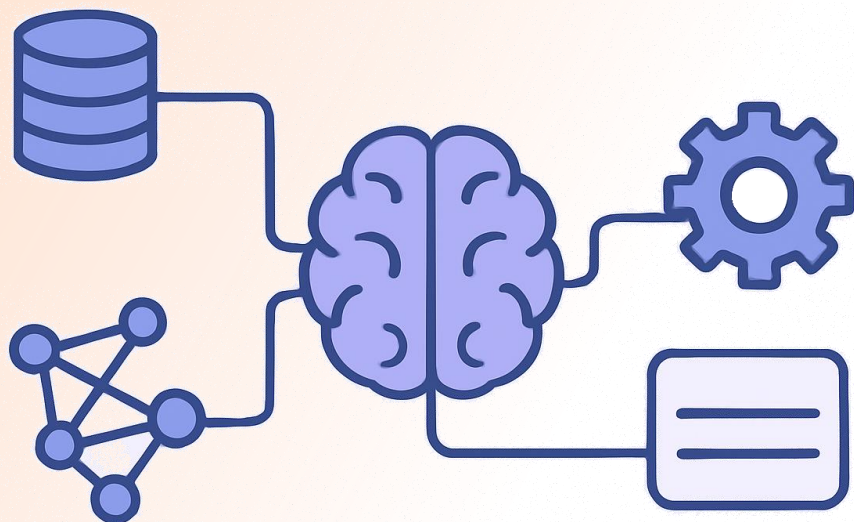
Error Rate Spike

Unexpected increase in error rates or critical system alerts

Data Drift

Input data distribution exceeding predefined thresholds from training data

DEMO



Summary

Takeaways

- **MLOps is Engineering, not just Science:** move from experimental notebooks to disciplined, scalable pipelines. MLOps is about turning a model from a "research project" into a reliable, high-value product.

- **Environment consistency is vital:** align training and serving environments using Docker and strict dependency management. This eliminates the "it works on my machine" problem and prevents logic mismatches in production.

- **Deploy Progressively, Fail Safely:** never "just deploy." Use Shadow or Canary releases to validate models on live traffic, and always set automated rollback triggers to protect the user experience.

- **Monitoring is never „done“:** ML models naturally degrade over time. Use inference tables and drift detection to ensure your system stays accurate and healthy long after the initial deployment.

Thank you!

Contact:



<https://www.linkedin.com/in/maciej-kepa>



<https://github.com/maciejkepa/ai-ml-in-practice>



<https://maciejkepa.dev/>

